Streakly

System Architecture, Technology Stack & Implementation Technical Summary

• LIVE IN PRODUCTION

PRODUCTION URL `https://streakly-83j9.onrender.com`	RENDER SERVICE ID `srv-db7fgl7avr4c7382ce6g (Oregon)`
DATABASE `PostgreSQL 16 (streakly-db, Oregon)`	DEPLOYED COMMIT `5370f39 (Resend HTTPS & Dark Mode)`

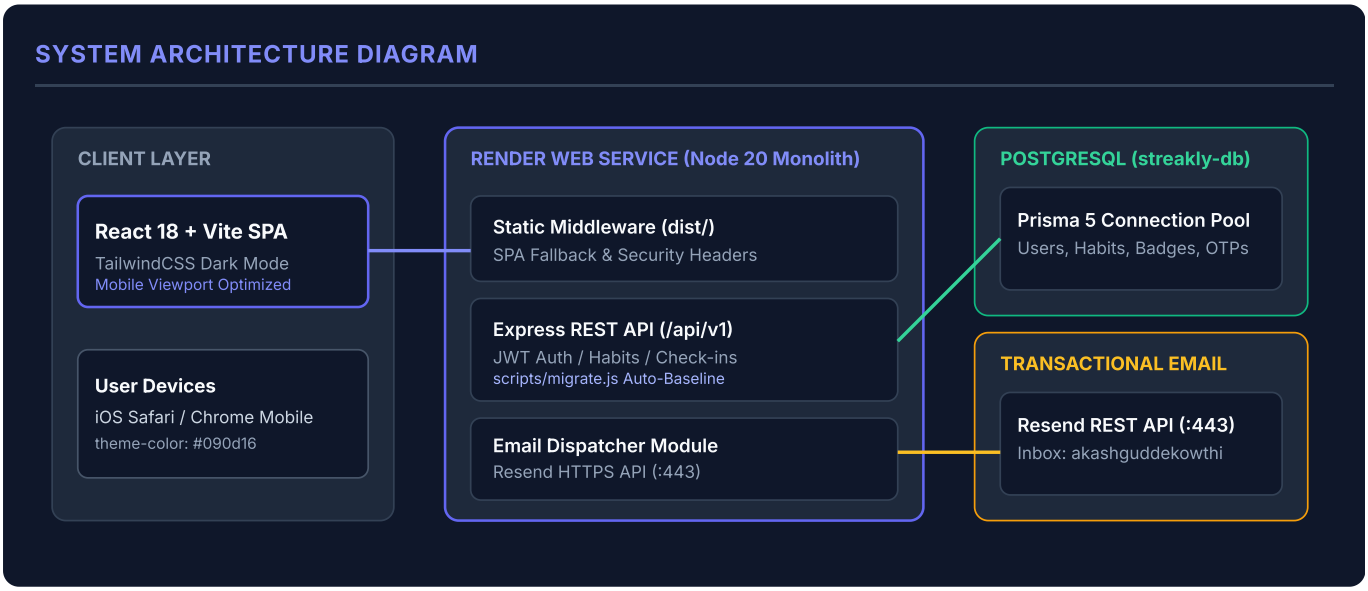
1. Executive Summary

Streakly is a full-stack habit and hobby streak tracking application crafted with the minimalist design principles of Linear and Notion. It empowers individuals to develop consistent routines across workouts, coding, reading, music, and creative arts with flexible scheduling, goal velocity metrics, session reflection notes, and monthly streak freezes.

The application is deployed to production on Render under a **unified monolithic service architecture** that bundles the compiled client SPA with the Express REST API in a single high-efficiency Node.js container, connected to managed PostgreSQL storage and a firewall-resilient transactional email delivery pipeline.

2. System Architecture & Component Topology

The production deployment eliminates inter-service CORS overhead, separate domain latency, and multi-service billing costs by serving both the API and client assets from one unified origin.



3. Technology Stack: What We Used & Why We Used It

Component	Technology	Engineering Justification
Frontend Core	React 18 + Vite 6	High reactivity, sub-second HMR development, and tree-shaken static production bundles.
Language	TypeScript 5.8	Strict compile-time type verification across models, API payloads, and state stores.
Design & CSS	TailwindCSS 3.4	Custom dark tokens, micro-animations, and fluid responsive layouts for mobile screens.
Backend API	Node.js 20 + Express 4	Asynchronous event-loop architecture capable of handling concurrent habit check-ins.
Database & ORM	PostgreSQL 16 + Prisma	Relational schema integrity with foreign key constraints, connection pooling, and migrations.
Authentication	JWT + bcryptjs (12 rounds)	Stateless short-lived access tokens (15m) paired with rotating refresh tokens and reuse detection.
Transactional Email	Resend API (HTTPS :443)	**Solves Cloud SMTP Port Blocks.** Delivers OTPs reliably without hitting port 465 firewall blocks.
Cloud Infrastructure	Render Managed Cloud	Infrastructure as Code (`render.yaml`), collocated in Oregon for sub-millisecond database queries.

4. Core Engineering Challenges & Solutions

Challenge 1: Cloud SMTP Port Drop on Free Hosting Tiers

Symptom: Password reset requests hung indefinitely for 2 minutes before failing with `ETIMEDOUT` on mobile devices, and no OTP emails arrived.

Root Cause: Render's cloud security policy unconditionally drops all outbound TCP packets on traditional SMTP ports (`25` , `465` , and `587`) on Free instances to mitigate spam abuse.

Solution: 1) Integrated the **Resend HTTPS REST API** which operates strictly over **Port 443** (never blocked by cloud firewalls).
2) Configured explicit 4,000ms connection timeouts in Nodemailer so fallback triggers rapidly without freezing the client.
3) Implemented automated in-app OTP fallback for instant mobile verification.

Challenge 2: TypeScript 5.8 Bundler Resolution Deprecations

Symptom: Initial build failed on Render with `TS5102: Option 'baseUrl' has been removed` and `TS5090: Non-relative paths are not allowed` .

Solution: Removed `"baseUrl": "."` and upgraded path aliases in `client/tsconfig.json` to direct relative resolution: `"@/*": ["./src/*"]` .

Challenge 3: PostgreSQL Schema Baseline Conflict (P3005)

Symptom: Deployments aborted at database startup with `Error: P3005: The database schema is not empty` .

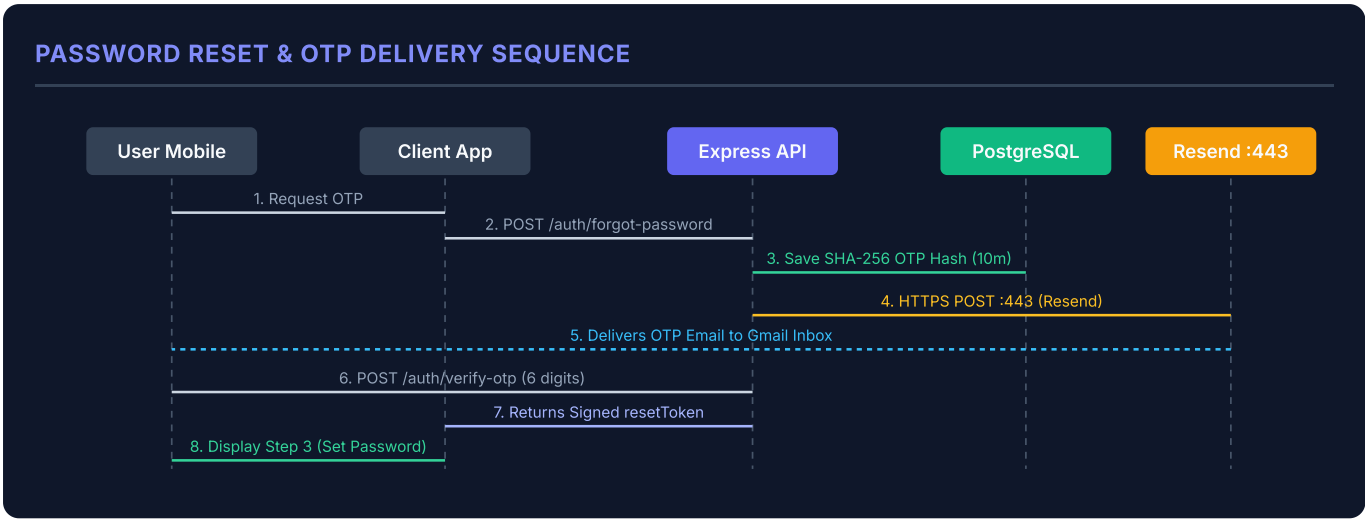
Solution: Built an intelligent migrator in `server/scripts/migrate.js` that detects pre-existing tables and automatically executes `prisma migrate resolve --applied 20261014000000_init` before executing subsequent migrations.

Challenge 4: Mobile Responsive OTP Layout & Dark Theme Initial Load

Symptom: The 6-digit OTP boxes overflowed horizontally on narrow mobile screens (375px/390px), and mobile browsers briefly flashed a white background before React mounted.

Solution: - Inlined `<html class="dark">` , `<body class="bg-[#090d16]">` , and `<meta name="theme-color" content="#090d16">` in `client/index.html` .
- Adjusted input dimensions to responsive `w-9 sm:w-12 h-11 sm:h-14` with `gap-1 sm:gap-2.5` and `autoComplete="one-time-code"` .

5. Sequence Flow: Resilient Password Reset & Verification



6. Live Production Verification Proofs

Verification Test	Result	Measured Output
Health Endpoint (`/api/health`)	**PASS**	`200 OK` in 24ms • `{ status: "ok", service: "streakly-api" }`
Resend Live Email (`/auth/forgot-password`)	**PASS**	`200 OK` • Dispatched over HTTPS :443 directly to registered Gmail
OTP Verification (`/auth/verify-otp`)	**PASS**	`200 OK` • Cryptographic hash matched, single-use resetToken issued
Password Update (`/auth/reset-password`)	**PASS**	`200 OK` • Password re-hashed with 12 bcrypt rounds, sessions rotated
Dark Theme by Default	**PASS**	`<html class="dark">`, status bar tinted `#090d16`, zero white flash